

Applied Human Computer Interaction (AHCI)

Asif Fazwani

Chapter 8 – Implementation Support

Programming Interactive Systems

Focus: How interactive systems are **actually built (coded)**

Shift from: Abstract design → **Real implementation**

Goal: Understand how interaction becomes **working software**

Designing a cool UI in Figma is easy but making it work in React or Flutter is the real challenge.

From Design to Coding

Earlier focus: **User perspective (UX design, flows)**

Now focus: **Programming perspective**

Developers must: Translate design → **machine instructions**

Elements Of Windowing Systems

Part of HCI that supports **WIMP interface**

Windows, Icons, Menus, Pointer

Focus here: **How it works for programmers (not just users)**

Example: Opening multiple apps on Windows 11 or macOS using different windows

Device Independence Windowing systems hide: (Keyboard differences, Mouse vs touch input, Display variations)

Programmers don't code for each device separately

Example: A web app running the same on: Laptop, Tablet, Smartboard in class

Elements Of Windowing Systems

Abstract Terminal Concept: Developers interact with a **virtual (abstract) device**

This device: Uses a **generic language** and Translates to real hardware commands.

Example: Apps built in Java run on different systems using JVM → no need to rewrite code for each device.

Imaging Model: Defines how graphics are displayed includes:
Shapes, Text, Images

Uses primitives (basic drawing commands)

Elements Of Windowing Systems

Text & Graphics Handling Text can be: Pixel-based (exact image) & Font-based (scalable text). Improves performance and flexibility

Example: Zooming text in Microsoft Word without losing quality.

Input in Imaging Models Imaging models also help with input ; Mouse click → pixel coordinates

Application decides: What action to take

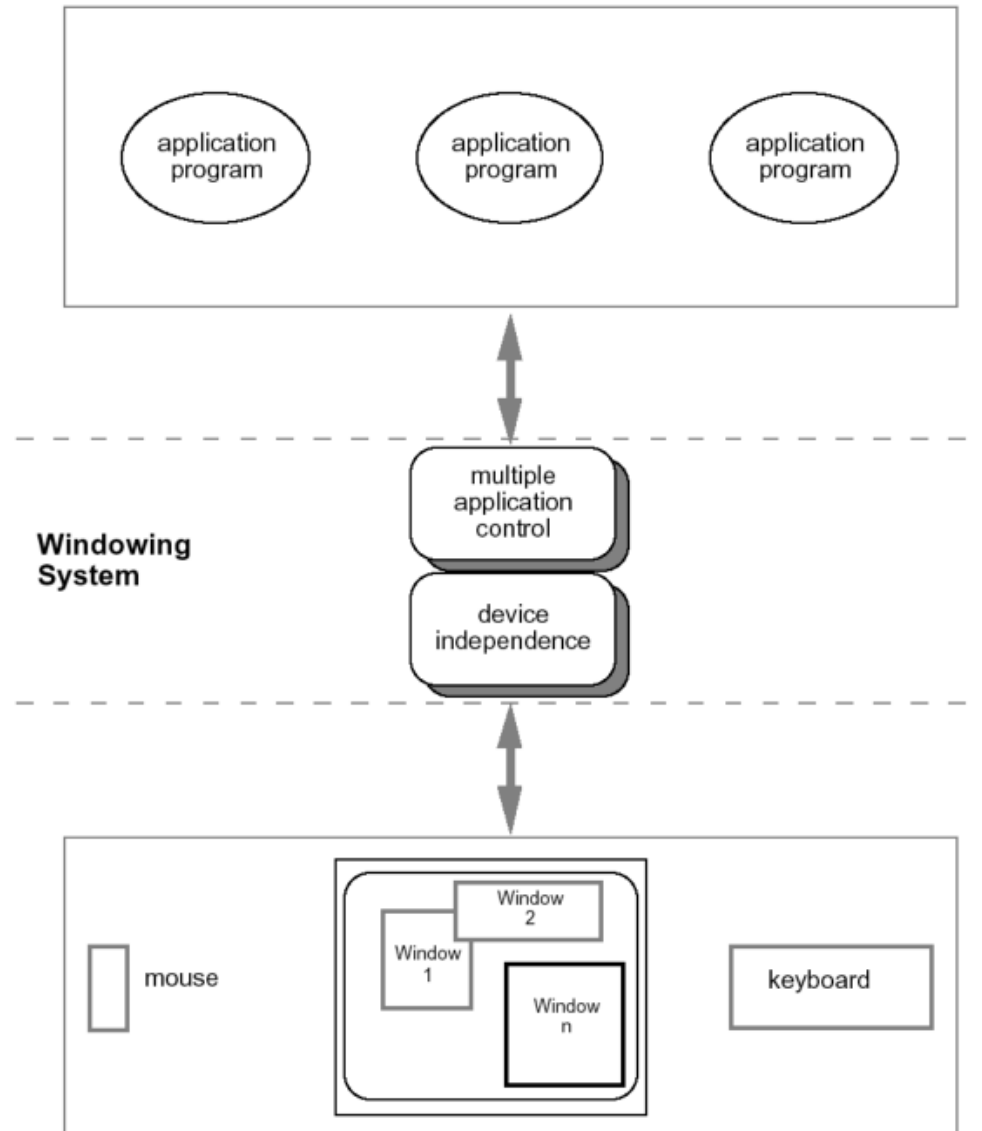
Example: Clicking a button in YouTube: System detects coordinates → App checks if it's on "Play"

Key Roles of Windowing Systems

1. Hardware Independence One app works on many devices

2. Multi-Application Management Run many apps simultaneously

Windowing systems = **"manager between apps and hardware"**



Architectures of windowing systems

3 main ways to design windowing systems:

- **Each application manages all processes (Inside each application)**
everyone worries about synchronization
reduces portability of applications
- **Management role within kernel of operating system (Inside operating system)**
applications tied to operating system
- **Management role as separate system (client-server)**
maximum portability

Architectures of windowing systems

Architecture 1: Each application manages all processes (Inside each application)

Each app manages:

- Input/output
- Multi-tasking
- Resource conflicts
- Problems:
 - Very complex
 - Low portability
 - Repeated effort

Analogy: Every app acting like its own mini operating system

Example: Imagine every app you install handles keyboard, mouse, and screen differently → chaos.

Architectures of windowing systems

Architecture 2: Management role within kernel of operating system (Inside operating system)

Window management handled by: Operating system

Benefits: Centralized control and Less burden on apps

Limitation: Apps tied to specific OS

Example: Apps built specifically for Windows OS may not easily run on Linux.

Architectures of windowing systems

Architecture 3: Client–Server Model

Window system runs as **separate program**

Apps = clients

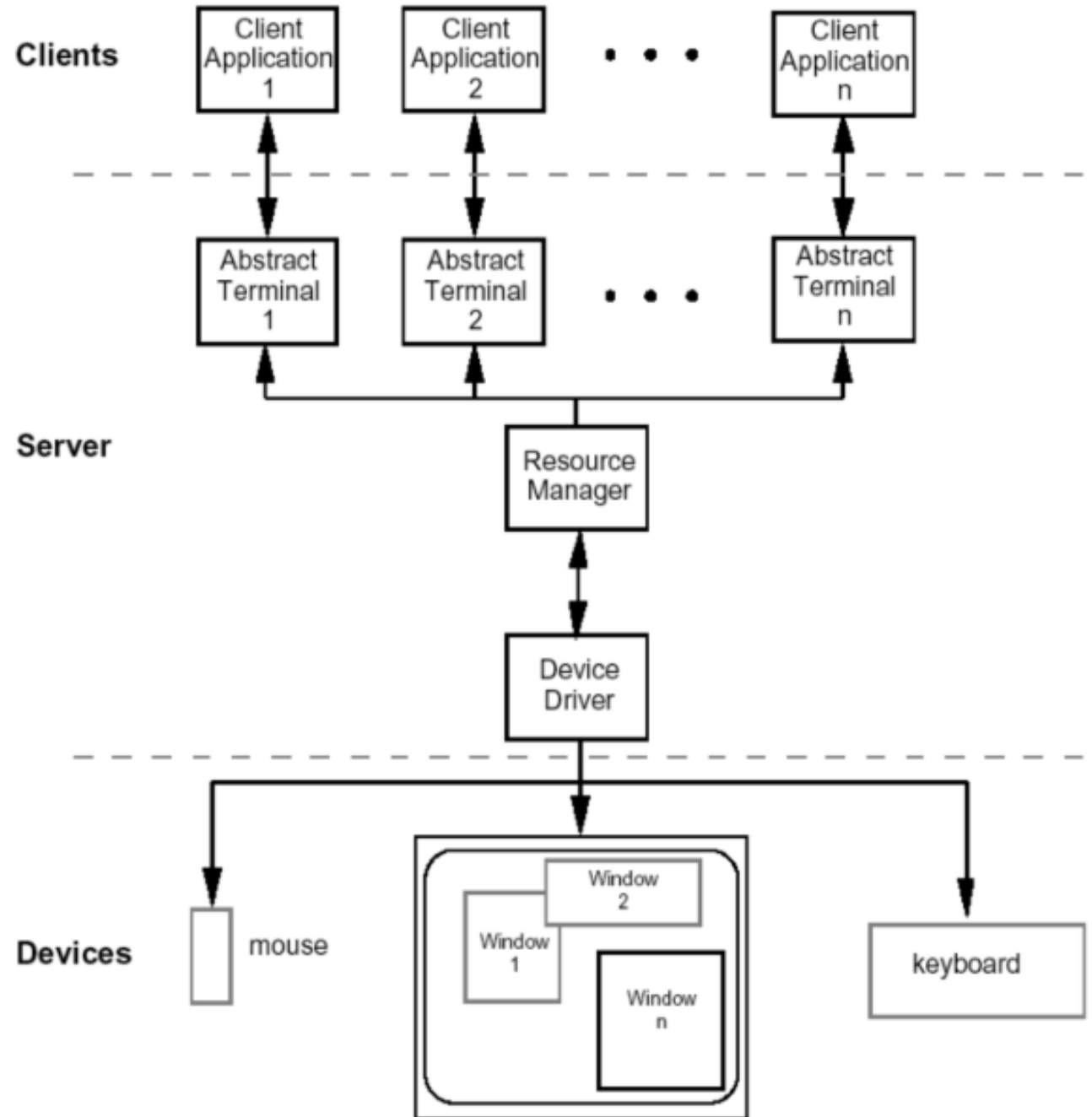
Window system = server

Communication via standard interface

Why best? High portability & Flexible across systems

Example: Browser apps connecting to cloud servers → similar idea of separation.

Client Server Architecture



X Window System

Developed at MIT also called: X or X11

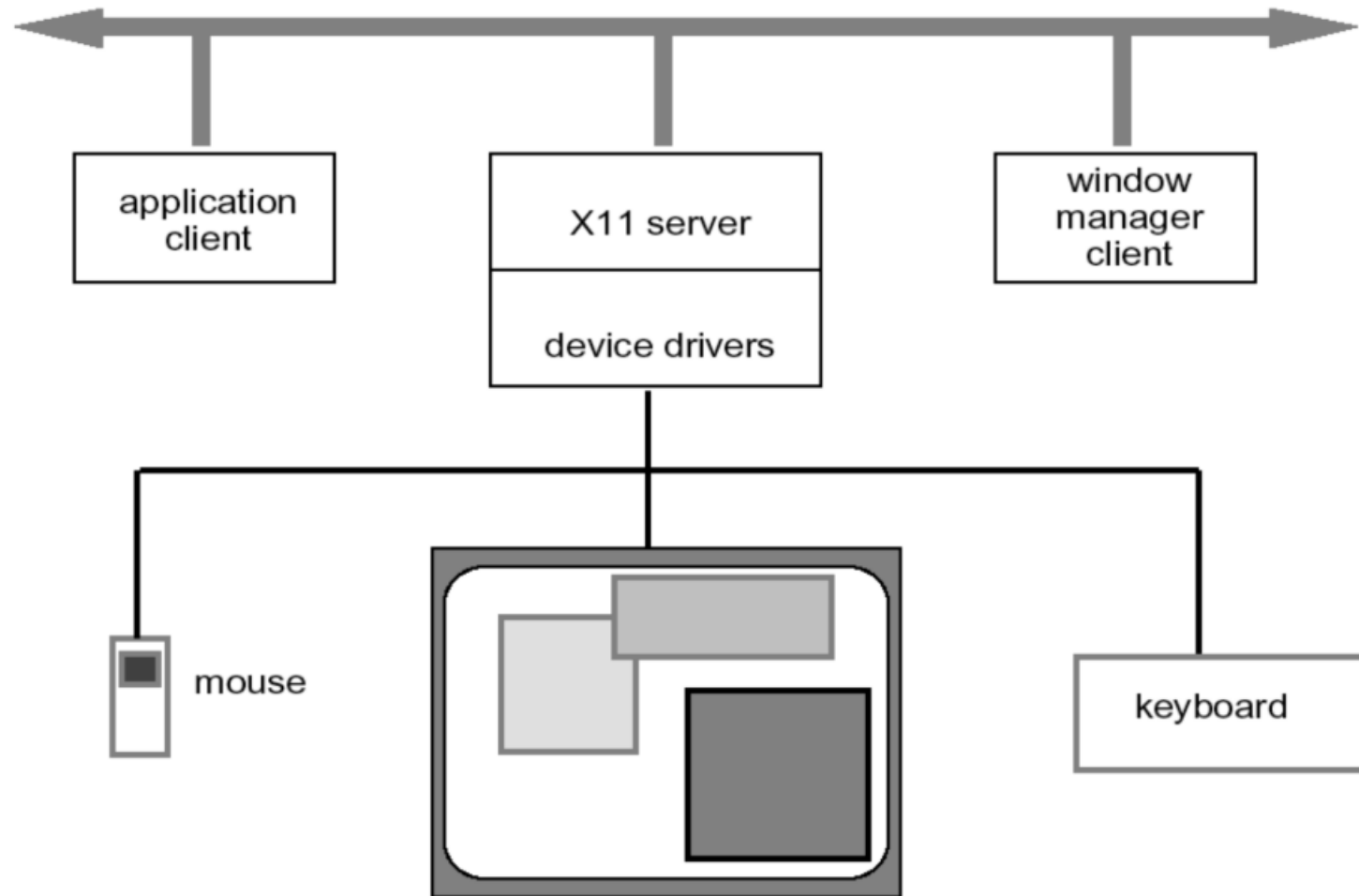
Based on: Client–server architecture & Pixel-based display mode

Why X Window System is Important

- Uses **network protocol**
- Works across:
 - Different computers
 - Different operating systems

Example: Running an app remotely (like cloud IDEs or remote desktops)

X Window System



Client–Server Communication in X

Clients send requests:

- Draw graphics
- Handle input

Server:

- Executes and manages display

Example: Like using Google Docs:

Your device = client

Google servers = processing

What X Server Does

Controls access to display

Handles screen operations

Distributes input events

Optimizes performance

Programming the application - Read-Evaluation Loop

Two Programming Paradigms

Two ways to manage control flow

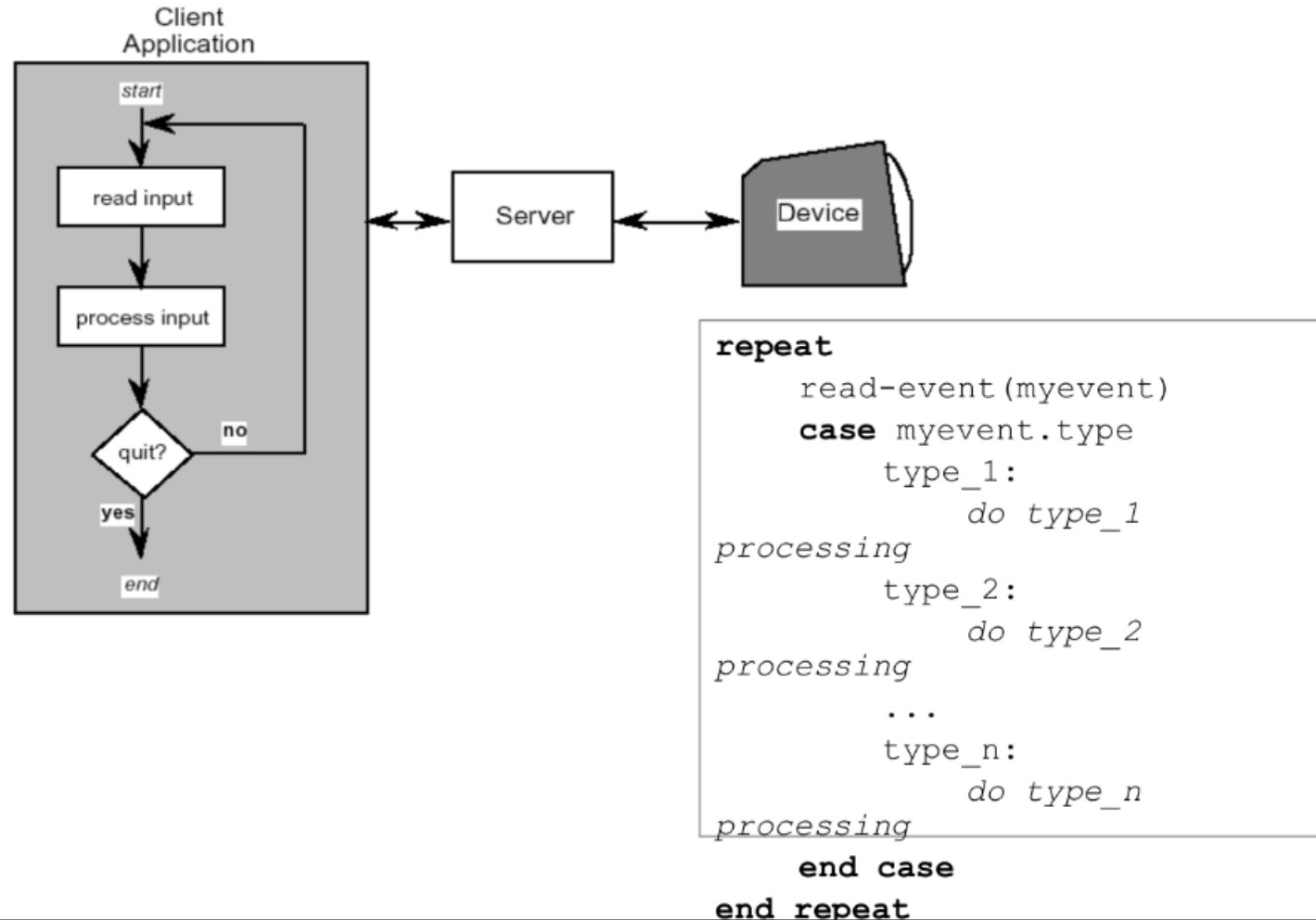
- Read-Evaluation Loop
- Notification-Based (Event-driven)

Read-Evaluation Loop: App continuously:

- Reads event
- Evaluates it
- Executes action

Loop runs forever

Event → Check type → Respond → Repeat



Programming the application - Read-Evaluation Loop

Application has:

Full control

- Must handle:
 - Every possible event

Example: A game built in Unity manually processes:

- Keyboard input
- Mouse clicks
- Game state updates

Programming the application - Notification-Based Paradigm

App does NOT control loop

A central notifier:

- Receives events
- Sends relevant ones to app

“Don’t call events → events call you”

Callback Mechanism

App registers:

- Events it cares about
- Functions (callbacks)

When event happens:

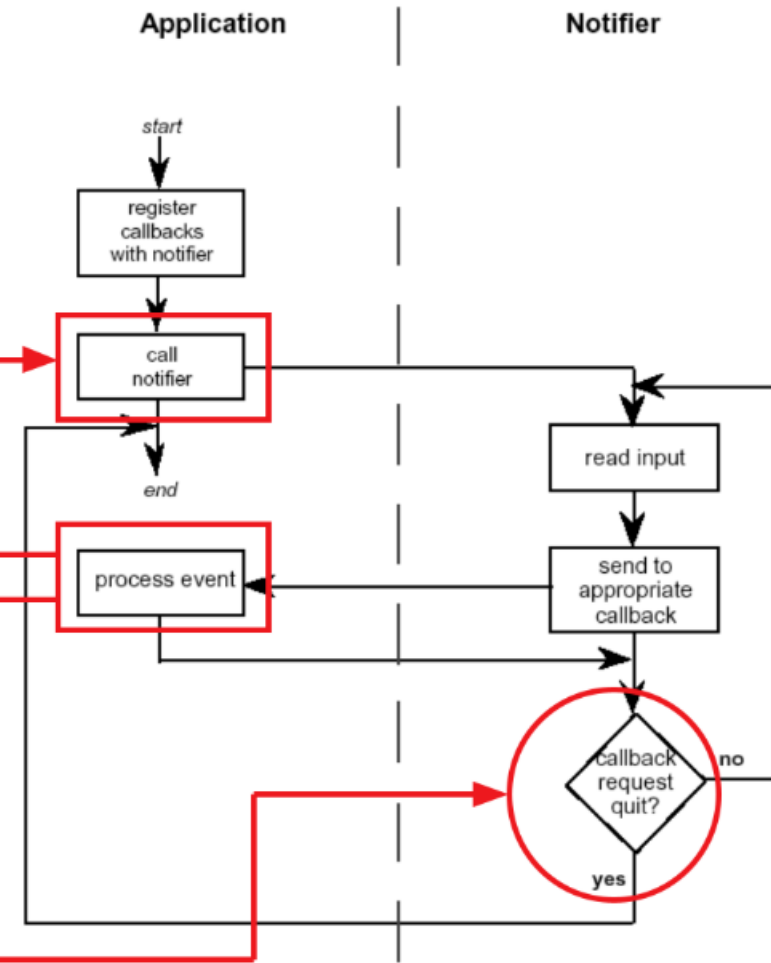
- Callback is triggered

notification-based

```
void main(String[] args) {  
    Menu menu = new Menu();  
    menu.setOption("Save");  
    menu.setOption("Quit");  
    menu.setAction("Save", mySave)  
    menu.setAction("Quit", myQuit)  
    ...  
}
```

```
int mySave(Event e) {  
    // save the current file  
}
```

```
int myQuit(Event e) {  
    // close down  
}
```



Programming the application - Notification-Based Paradigm

Benefits of Notification Model

- Less work for developer
- Cleaner code
- No need to process every event

Drawback – Less Control

- App loses: Direct control overflow
- Harder to: Interrupt system behavior

Using Toolkits & Interaction Objects

Interaction Objects UI elements = **interaction objects** (widgets, gadgets, techniques)

Combine: **Input + Output** intrinsically linked

Clicking a button in Instagram → **visual feedback + action happens instantly**

Abstraction with Toolkits Toolkits provide: **Higher-level abstraction**

Hide: Low-level input/output handling

Using a button component in React instead of coding mouse events manually

Using Toolkits & Interaction Objects

Programming with Widgets Developers build apps using: Predefined interaction objects Focus on: Behavior, not hardware

Example: Forms in Google Forms built from ready-made fields

Consistency & Generalizability Toolkits ensure: Same **look and feel**
Improves: Usability & Learnability

Example: Buttons look similar across apps in Android

Object-Oriented Support Based on OOP concepts: Classes → Widgets & Objects → Instances

Supports: Reusability & Extensibility

Example: Creating multiple styled buttons from one base component

User Interface Management Systems (UIMS)

User Interface Management System It provides: Higher-level support beyond toolkits and helps in: **Design, Implementation, and Management of UI.**

Goals of UIMS

Separate: **Application logic (what system does) & Presentation (how it looks)** Provide: Better structure for UI systems

Key Functions of UIMS

- Conceptual architecture
- Techniques for linking UI + application
- Runtime support (manage interaction)

User Interface Management Systems (UIMS)

Separation of Concerns Divide system into:

Application (logic) Presentation (UI) Dialog control (communication)

In Yango: App logic = ride matching, UI = map + buttons Dialog = interaction between both

Benefits of Separation

- **Portability:** Same app, different platforms
- **Reusability:** Reuse components
- **Multiple Interfaces:** Same logic, different UI
- **Customization:** Change UI without changing logic

Example: Spotify: Same backend & Different UI on mobile, desktop, web

User Interface Management Systems (UIMS)

Dialog Control

Acts as: “Bridge” between UI & application

Manages: Communication flow

Idea: User → UI → Dialog → App → UI

Dialog Control Placement

Read-evaluation loop Inside application

Notification-based Outside application

Insight: UIMS usually use **external dialog control**

Seeheim Model (Classic UIMS Architecture)

3 components: Presentation, Dialog Control, Application Interface

Presentation (lexical level) - UI elements

Dialog Control (syntactic level) - Interaction logic

Application (semantic level) - Core functionality

Example: Form submission: UI = form, Dialog = validation, App = database save

Limitations of Seeheim Model

No clear guidance for future design

Mixes: Conceptual ideas & Implementation details

Doesn't scale well for complex systems

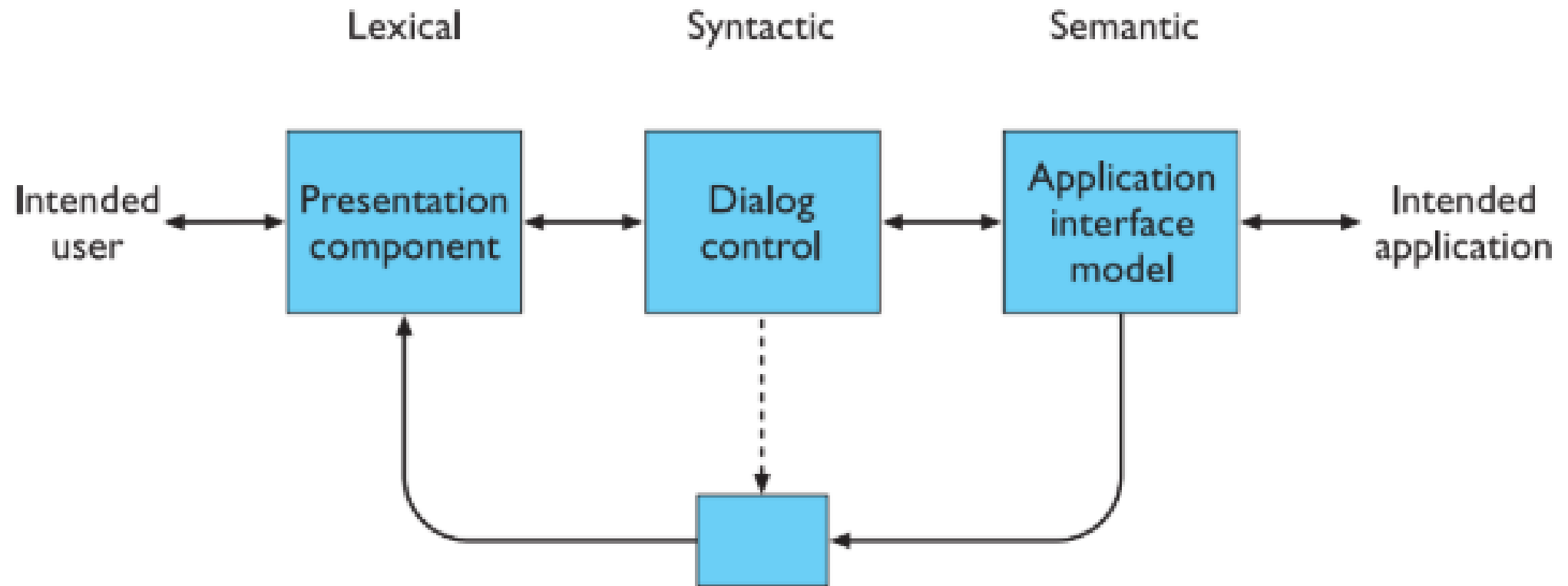


Figure 8.10 The Seeheim model of the logical components of a UIMS

Conceptual vs Implementation

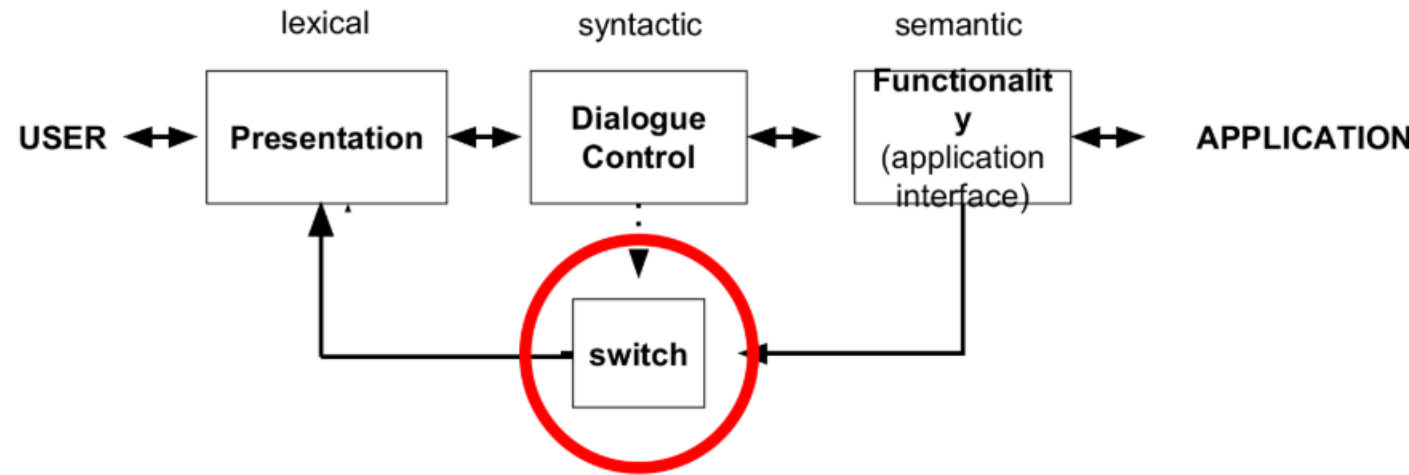
Conceptual model = how we think about system structure

Implementation model = how we actually build it

Why It Changed Thinking

Before Seeheim: UI + logic mixed together

After Seeheim: Clear separation mindset



The “**Switch**” Lower box in diagram = **implementation shortcut**

Allows:

- Direct communication
- Skips dialogue control

Purpose: Improve efficiency

Why “Switch” is NOT Conceptual

- Not part of: Clean architecture thinking
- Only added for: Practical coding needs

Conceptual model = clean

Implementation = messy reality

Semantic feedback

Different kinds of feedback:

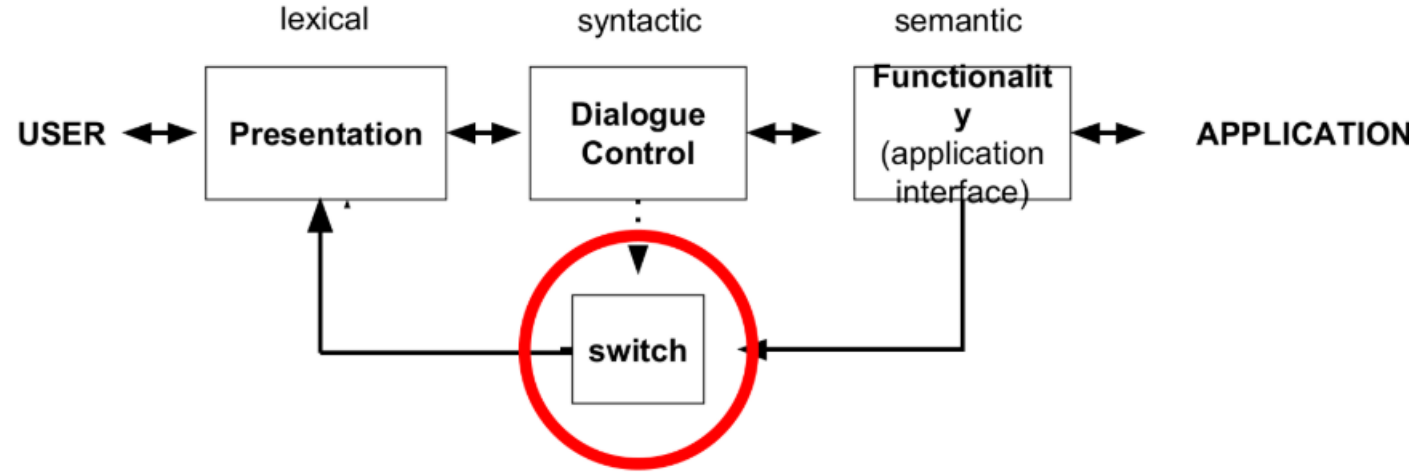
- lexical – movement of mouse
- syntactic – menu highlights
- semantic – sum of numbers changes

Semantic feedback often slower

- use rapid lexical/syntactic feedback

But may need rapid semantic feedback

- freehand drawing
- highlight trash can or folder when file dragged



The Bypass/Switch

What is the “Bypass / Switch”?

A **direct communication path** between:

- Presentation (UI)
- Functionality (application logic)

Skips Dialogue Control layer

Also called: Shortcut / optimization path

Why Does the Bypass Exist?

Main reason: **Rapid semantic feedback**

Improves:

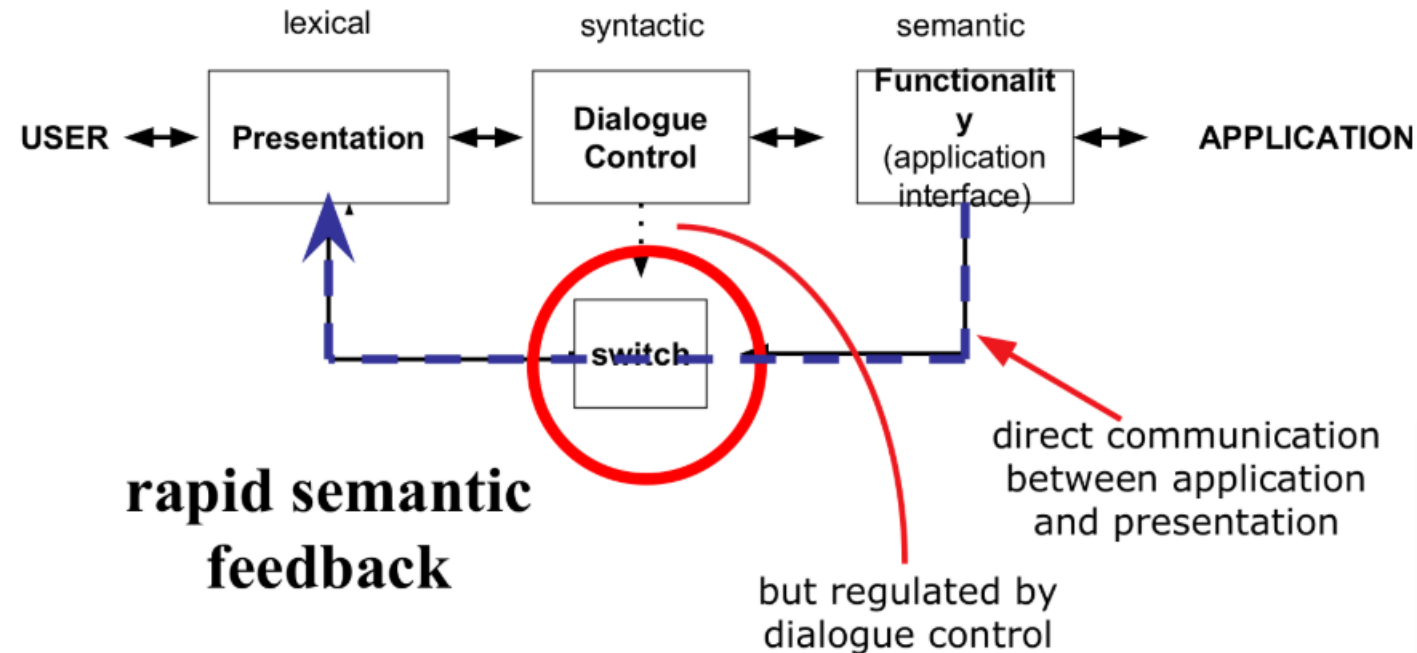
- Speed
- Responsiveness

Typing in Google Docs:

Text appears instantly

No visible delay through full control flow

the bypass/switch



Normal Flow vs Bypass

Normal (Conceptual Model): User → Presentation → Dialogue Control → Functionality

Bypass (Implementation Shortcut): User → Presentation ↔ Functionality (direct)

Dialogue control is partially skipped

What “Rapid Semantic Feedback” Means

System responds: Immediately & Meaningfully

Example: Live reactions (likes, typing indicators) in WhatsApp

Bypass is: **Allowed in implementation** But: Still **logically controlled** by dialogue layer

Why Not Always Use Bypass?

Breaks: Clean separation of concerns

Leads to: Harder debugging & Poor maintainability

Insight: Fast ≠ always good design

Conceptual vs Implementation View

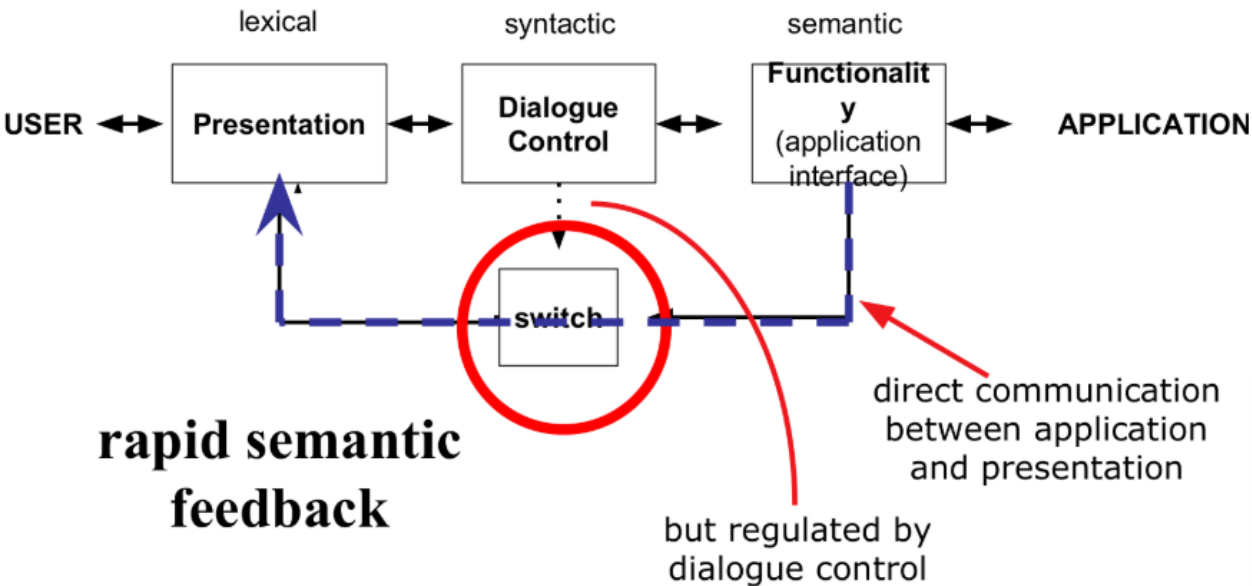
Conceptual (Seeheim ideal): Always go through Dialogue Control

Implementation (real systems): Use bypass for performance

This is why the “switch” exists

The Bypass/Switch

the bypass/switch



Monolithic vs Component-Based Design

Monolithic (Seeheim style):

Few large components e.g., Presentation, Dialogue, Application

Component-based: Many small reusable pieces

Problem with Monolithic Design It is hard to:

1. Modify 2. Scale 3. Reuse parts and Changes affect entire system

Shift from Seeheim to Component Models

- Seeheim: Conceptual, large blocks
- Modern systems: Break into **smaller units**

It leads to:

- Better flexibility
- Scalable design

MVC Model

A component-based architecture

Model → data & logic **View** → UI display **Controller** → input handling

Model (Logic Layer)

Stores: Internal state & Data , Independent of UI

Example: User profile data in Instagram

View (Presentation Layer)

Displays: Data to user, it can have: Multiple views for same model

Example: Same playlist shown differently in Spotify

Controller (Interaction Layer)

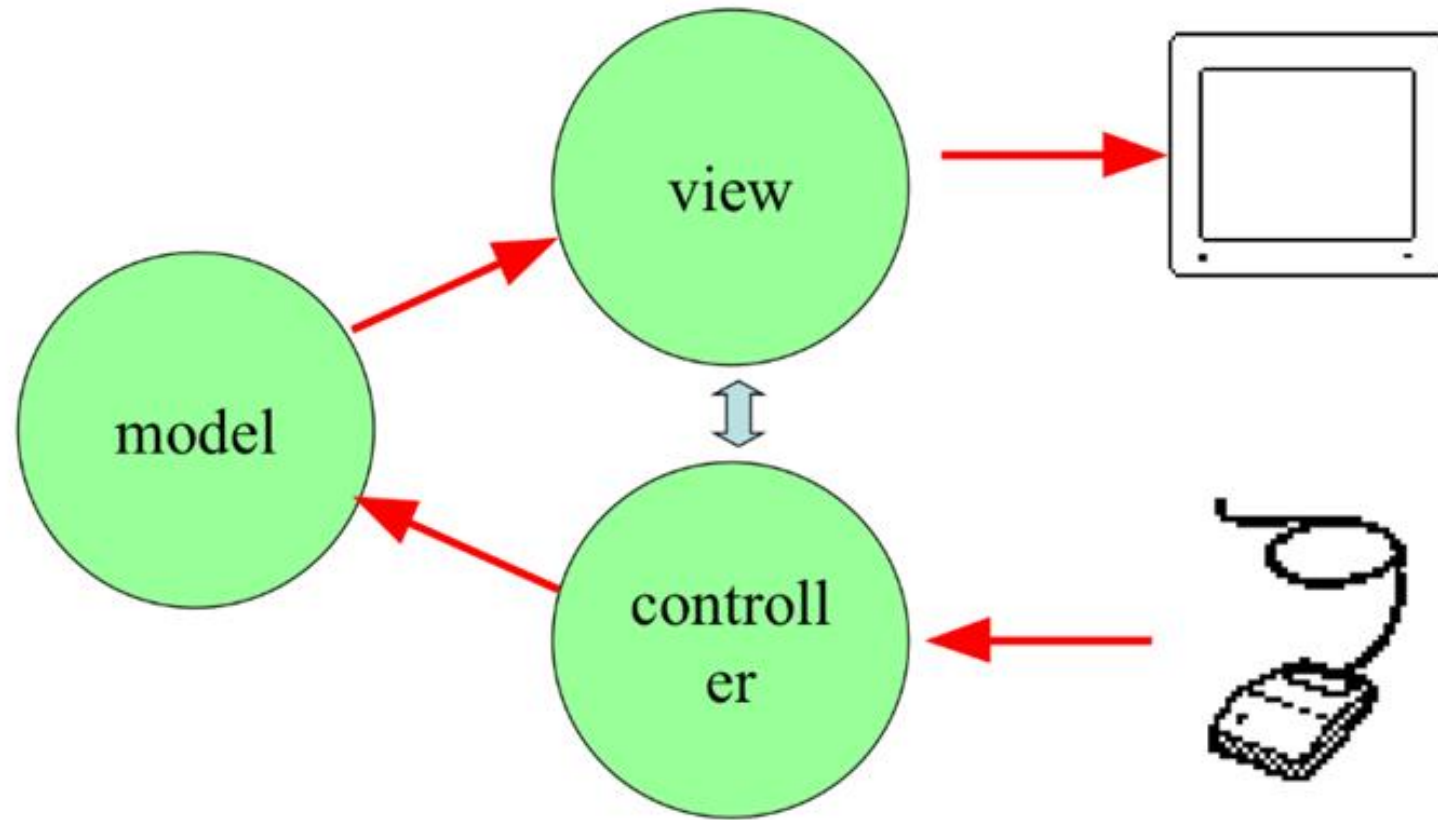
Handles: User input

Updates: Model or View

Example: Clicking “Like” triggers controller logic

MVC

model - view - controller



MVC Model

Why MVC Fits Modern Apps

Works well with: Object-oriented toolkits

Supports: Reusable UI components

Example: Frameworks like Django use MVC-like patterns

MVC issues

MVC is largely pipeline model: input → control → model → view → output

but in graphical interface input only has meaning in relation to output

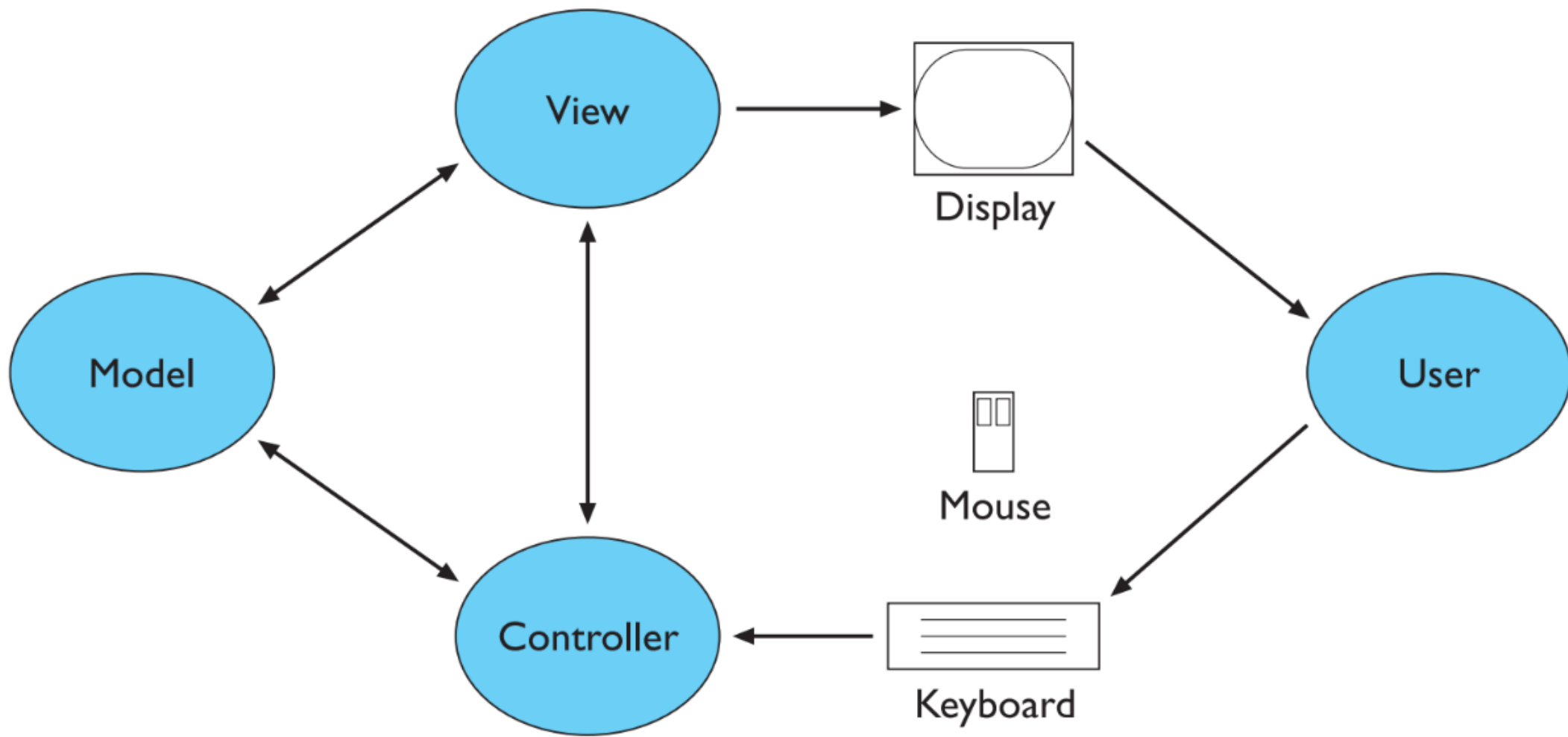
e.g. mouse click

need to know what was clicked

controller has to decide what to do with click

but view knows what is shown where!

In practice controller 'talks' to view
separation not complete



PAC Model

A **component-based architecture** closer to: Seeheim Model

Breaks system into **small cooperating units (agents)**

PAC Components Each PAC unit (agent) has 3 parts:

- **Abstraction**
 - Data + logic (state)
- **Presentation**
 - Input + Output
- **Control**
 - Communication + coordination

PAC Model

Abstraction (Logic Layer) Represents: Internal state & Application logic

Example: User profile data in Instagram

Presentation (UI Layer) Handles: Display & User input

Example: UI elements (buttons, feeds, icons) & Unlike MVC: Input + output are **combined here**

Control (Key Feature) Mediates between: Abstraction ↔ Presentation

Manages: Communication & Synchronization

Also connects: Multiple PAC components

PAC Model

Hierarchy in PAC

PAC uses: **Multiple agents (components)**

Organized in: Hierarchy

Example: App UI: Main screen, Sidebar, Chat window → Each = PAC component

Communication in PAC

- Control components: Talk to each other
- Enables: Coordination across UI parts

Example: Notification updates multiple UI sections at once

PAC vs MVC

Why PAC is “Cleaner”

- Clear responsibilities
- Better separation
- Explicit control layer

Easier to manage:

- Complex systems
- Multiple views

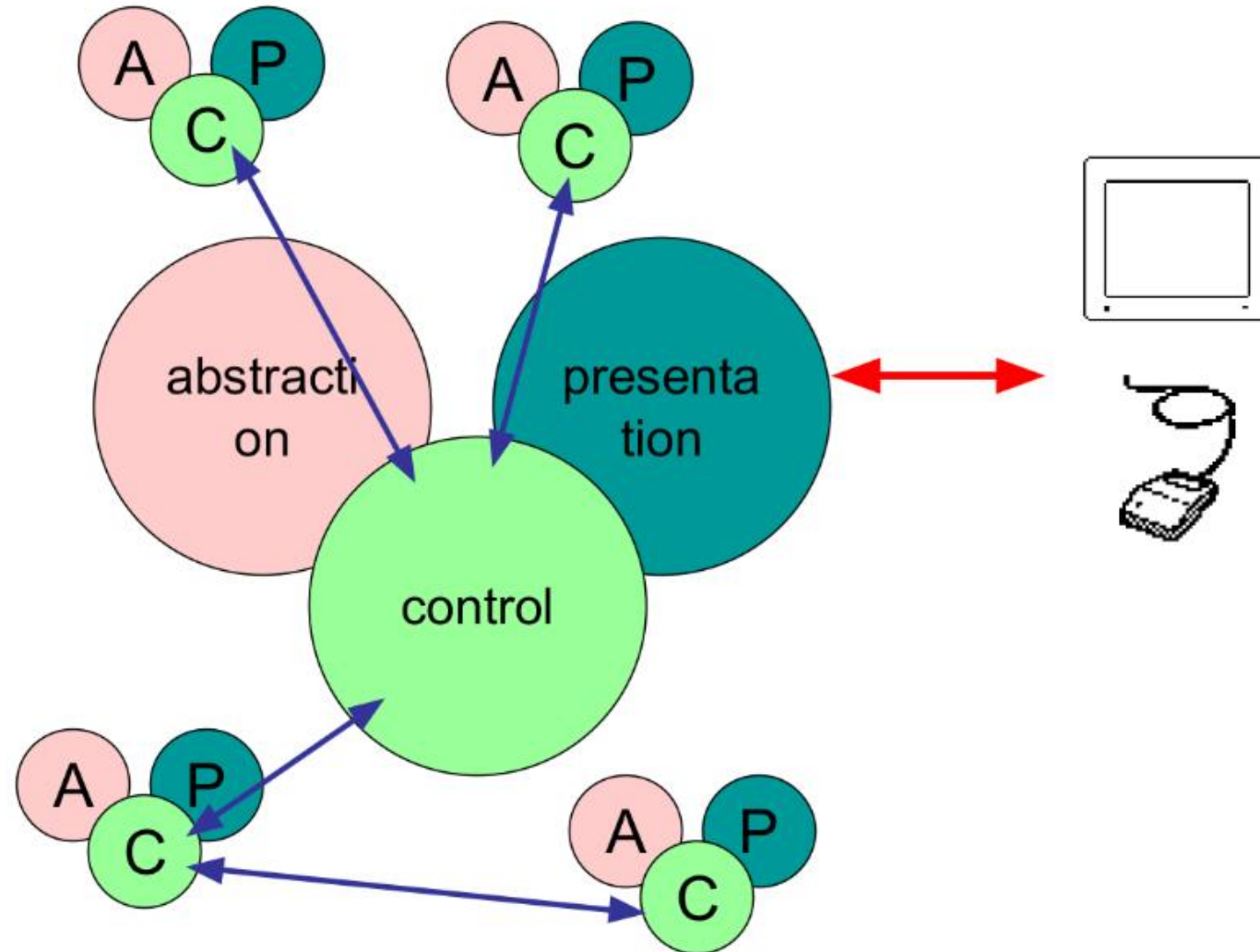
Why MVC is More Popular

- Simpler to understand
- Easier to implement
- Widely supported

Example: Frameworks like Java Swing use MVC-like patterns

PAC

presentation - abstraction - control



User Interface Management Systems

Implementation Considerations

Menu Networks (Core Technique) Dialog represented as: **Menus → Submenus → Actions**

Each menu has Represents possible user choices

How Menu Networks Work User selects option → system: Moves to next menu Or performs action Flow: Menu → Selection → Next screen

Graphical Menus: Menus not limited to text It can be: Buttons, Icons visual elements

Real Example – HyperCard Concept Entire system = network of screens

Each click: Moves to another “card”

Early version of modern UI navigation

Advantages of Menu Networks Easy to Understand & Implement ; good for Structured navigation systems

Limitations Not flexible for: Complex interactions It can become rigid and hard to scale

Implementation Considerations

Grammar Notations: Dialog is modeled like a **language**

Uses: Rules of actions & responses. These are good for describing command-based interfaces, but are not so good for more graphically-based interaction techniques.

State transition diagrams can be used as a graphical means of expressing dialog. The difficulty with these notations lies in linking dialog events with corresponding presentation or application events.

Login Flow

State 1: Start → **State 2:** Enter Username & Password → **State 3:** Validate Credentials

State 4A: Login Success **State 4B:** Error Message

Transitions: Start → Enter Details (user opens login page)

- Enter Details → Validate (user clicks login)
- Validate → Success (correct credentials)
- Validate → Error (wrong credentials)

Implementation Considerations

Event languages are similar to grammar notations, except that they can be modified to express directionality and support some semantic feedback. Event languages are good for describing localized input–output behavior in terms of production rules.

Form Validation Rule: If user enters invalid email then show error message

Example: Typing wrong email in Gmail signup → “Enter a valid email address”

Declarative Languages: A declarative approach concentrates more on describing how presentation and application are related. This relationship can be modeled as a shared database of values that both presentation and application can access. Declarative languages, therefore, describe what should result from the communication between application and presentation, not how it should happen in terms of event sequencing.

Example: Shopping Cart Total Rule (declarative style): **Total Price = Quantity × Price**

No steps defined (no “when click → update”) Just a relationship

Implementation Considerations

Constraints can be used to make explicit the connection between independent information of the presentation and the application.

Bank Balance Display - Constraint: Displayed Balance = Actual Account Balance in NayaPay Send money → balance updates instantly

Graphical Specification These techniques allow the dialog specification to be programmed graphically in terms of the presentation language itself. This technique can be referred to as programming by demonstration since the programmer is building up the interaction dialog directly in terms of the actual graphical interaction objects that the user will see.

Drag-and-Drop UI Design designer builds UI by: Dragging buttons, forms, images

Drift of Dialogue Control

What is “Drift”? Shift in: Where dialog control happens, Control moved across layers

Internal Control Dialog handled: Inside application

Example: Read–evaluation loop, App controls everything

External Control Dialog handled: Outside application Separate component manages interaction

Example: Event notifier systems

Presentation Control Dialog defined: In UI layer itself Interaction built directly into visuals

Example: Drag → link → define behavior